

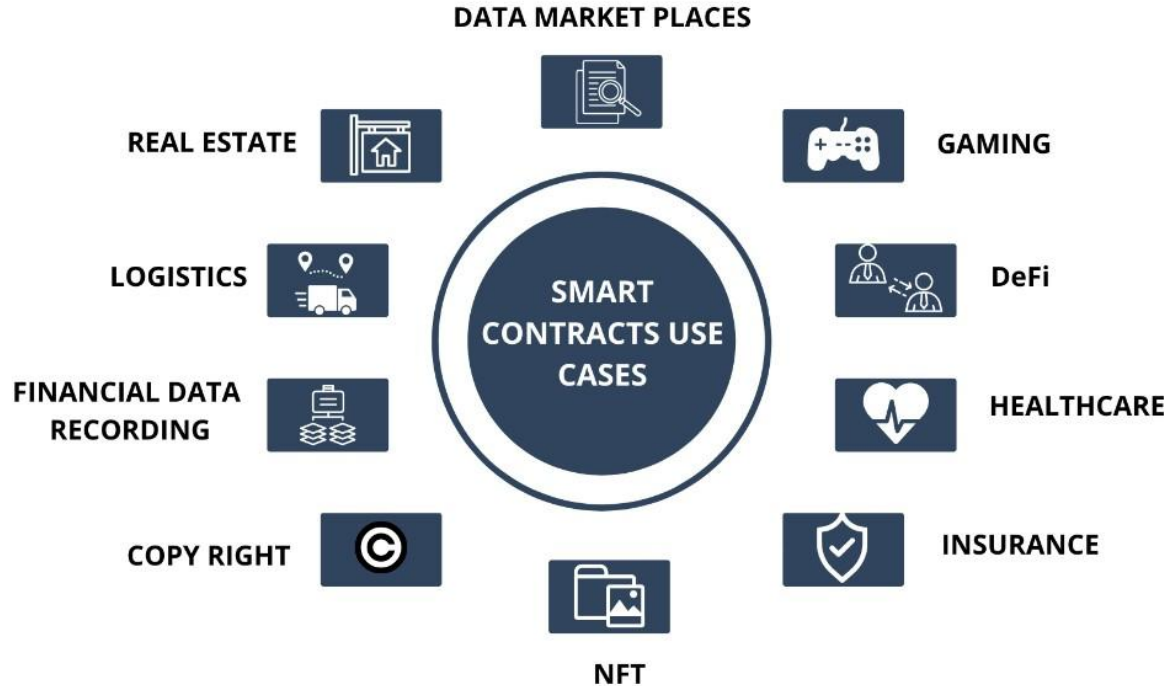


Towards Secure Smart Contracts: A Deep Learning Approach for Detecting Security Threats

Ph.D. Thesis Defence by **Tamer Abdelaziz**

National University of Singapore

Application of Smart Contracts



Advantages that pose challenges for security

- **Advantages & Challenges:**

- **Transparency:** Visible to everyone, including bad actors (**attackers**).
- **Immutability:** Tamper-proof security, but fixing code **vulnerabilities** later is tricky.
- **Valuable Assets:** Treasure (crypto, tokens, secrets) for users, but target for **attackers**.

- **Hidden Code, Hidden Risks:**

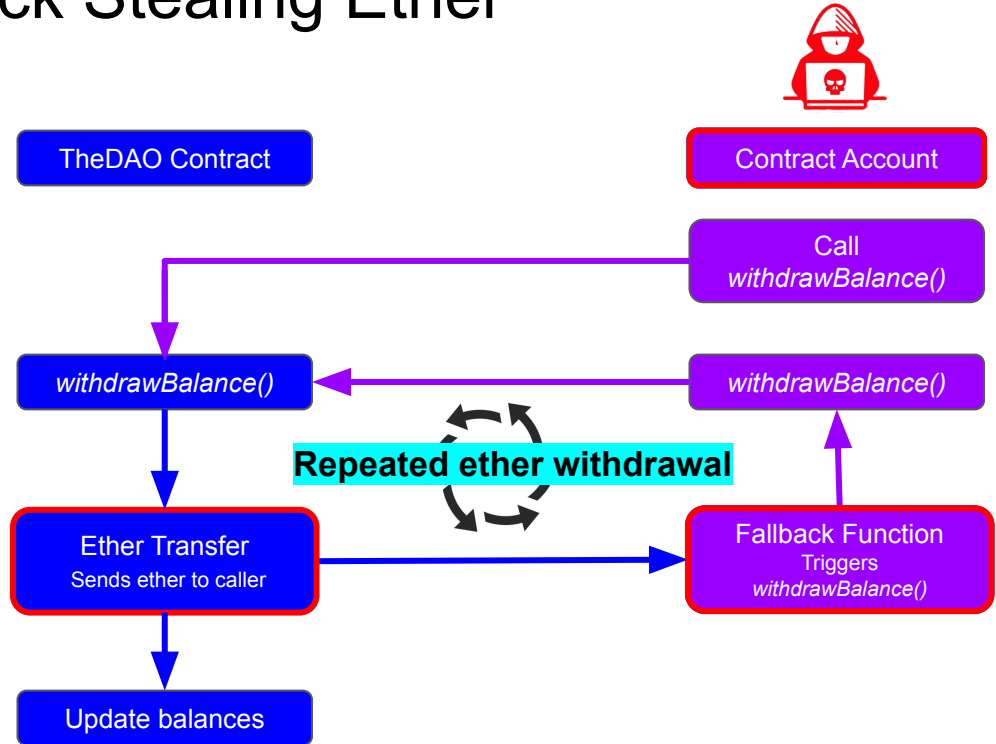
- Only **1%** of contracts reveal their **source code**, hindering vulnerability detection.
- **Bytecode** analysis is crucial to secure closed-source contracts and mitigate threats.
- **Auto-exploit** demos help developers build stronger contracts.

- **Solution:** Invest in bytecode analysis tools to secure contracts and protect valuable assets.

Example: Reentrancy Attack Stealing Ether

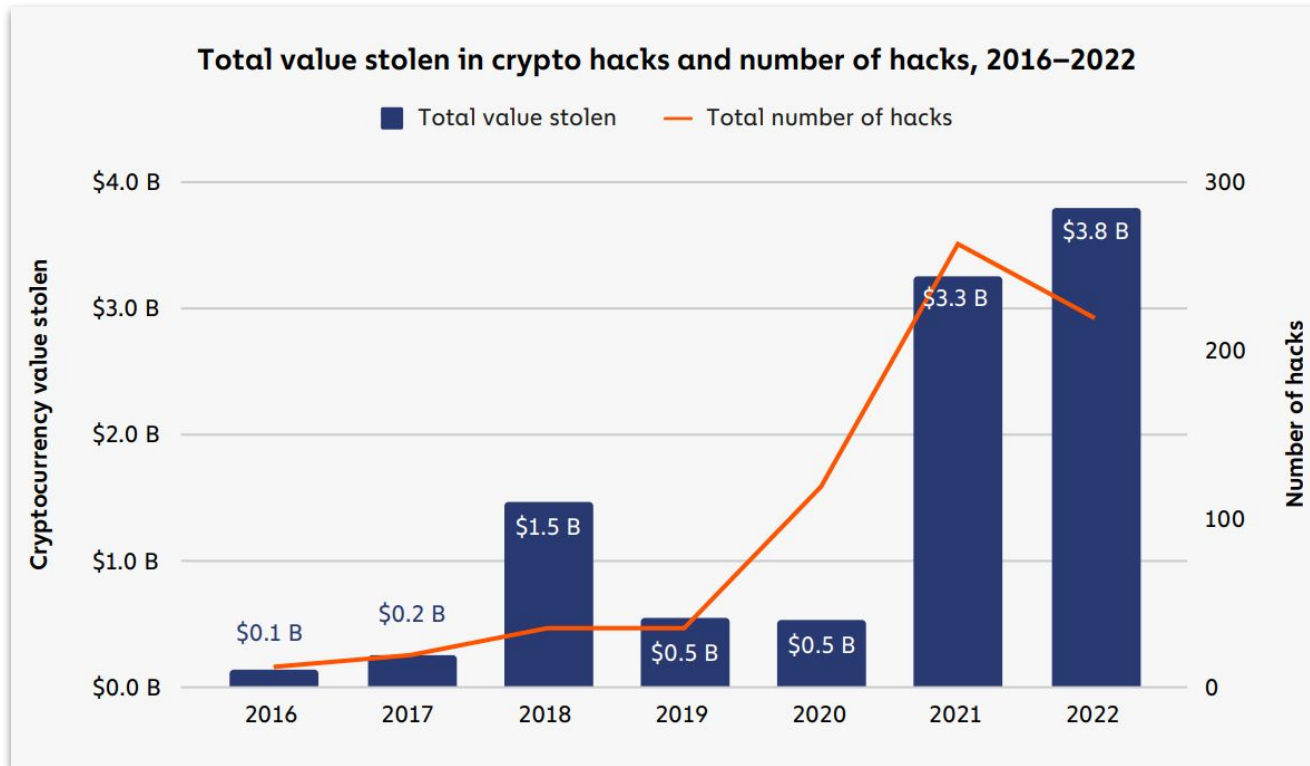


```
contract TheDAO {  
  
    mapping(address => uint) balances;  
  
    function deposit() public payable {  
        uint amount = balances[msg.sender];  
        balances[msg.sender] = amount + msg.value;  
    }  
  
    function withdrawBalance() public {  
        if (msg.sender.call.value(balances[msg.sender]))  
        { balances[msg.sender] = 0; }  
    }  
}
```



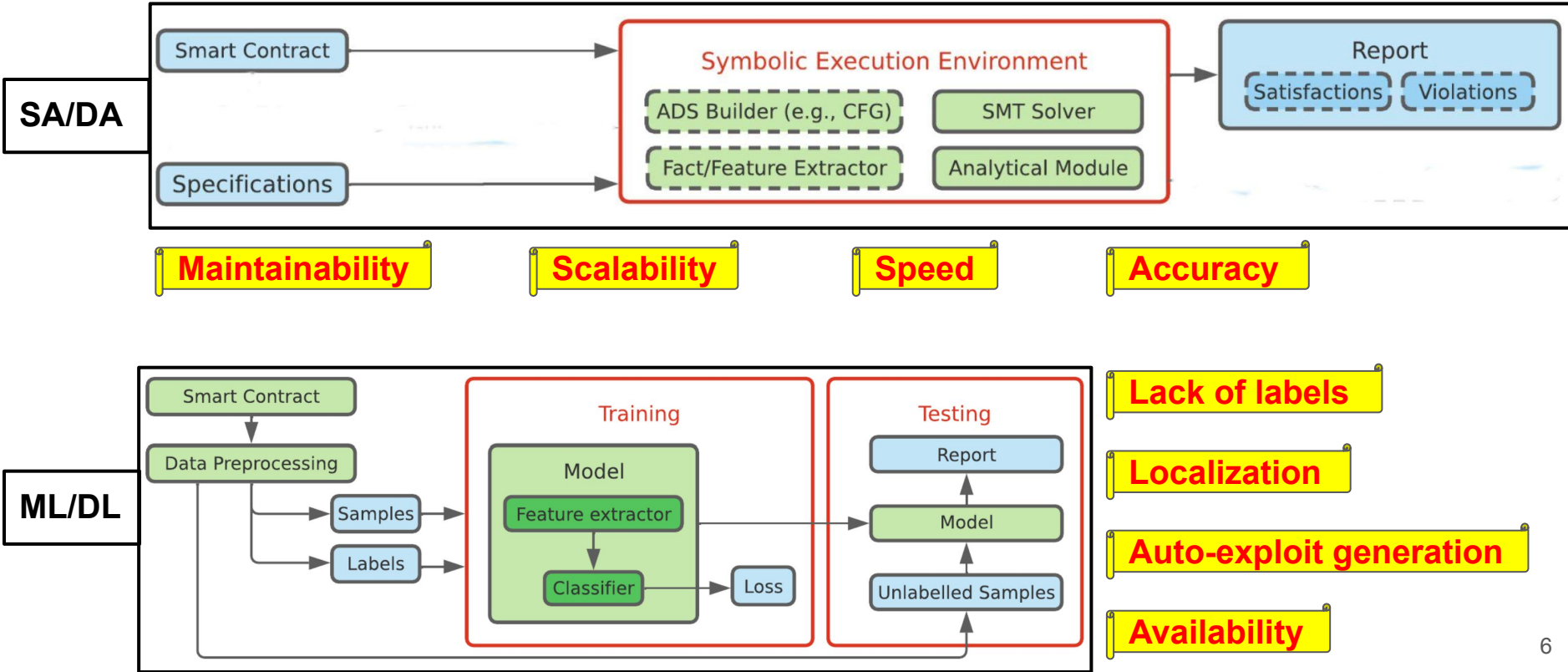
- TheDAO Attack (2016): **3.6** million ether stolen.
- Reentrancy remains a persistent issue in smart contracts to this day.

Vulnerability in Cryptocurrency are very Expensive



Source: The Chainalysis 2023 Crypto Crime Report

Vulnerability Analyzer: Real-World Roadblocks



Introducing....

DLVA : Deep Learning Vulnerability Analyzer
SCoolS : Smart Contract Learning (Semi-supervised)

Proposal 1: DLVA

[USENIX Security 23]

Vulnerability Detection

Maintainability

Scalability

29 Vulnerabilities

Accuracy

Speed

Availability

Proposal 2: SCoolS

[IEEE BCCA 23]

Exploit Detection

Lack of labels

Localization

1 Vulnerability & Auto-exploit

Ethical Disclosure

- **Challenge:**

Blockchain vulnerability disclosure is a unique challenge; active attackers seek vulnerabilities, and anonymity complicates alerting participants to potential threats. [Commun. ACM'20]

- **Our Solution:**

- Aligned with USENIX's Research Ethics Committee feedback, we prioritized responsible vulnerability disclosure, addressing and mitigating real-world impacts before releasing both tools (DLVA and SCoolS).
- Temporarily closed for safety enhancements, both tools are now reopened, empowering the community to contribute to a more secure blockchain ecosystem.
- Note: Only SCoolS auto-exploit generator is not available publicly until 25 Feb 2024.

DLVA: Deep Learning Vulnerability Analyzer

Proposal 1: DLVA

[USENIX Security 23]

Vulnerability Detection

Maintainability

Scalability

Speed

Accuracy

Availability

29 Vulnerabilities



Tamer Abdelaziz and Aquinas Hobor. "[Smart Learning to Find Dumb Contracts](#)".

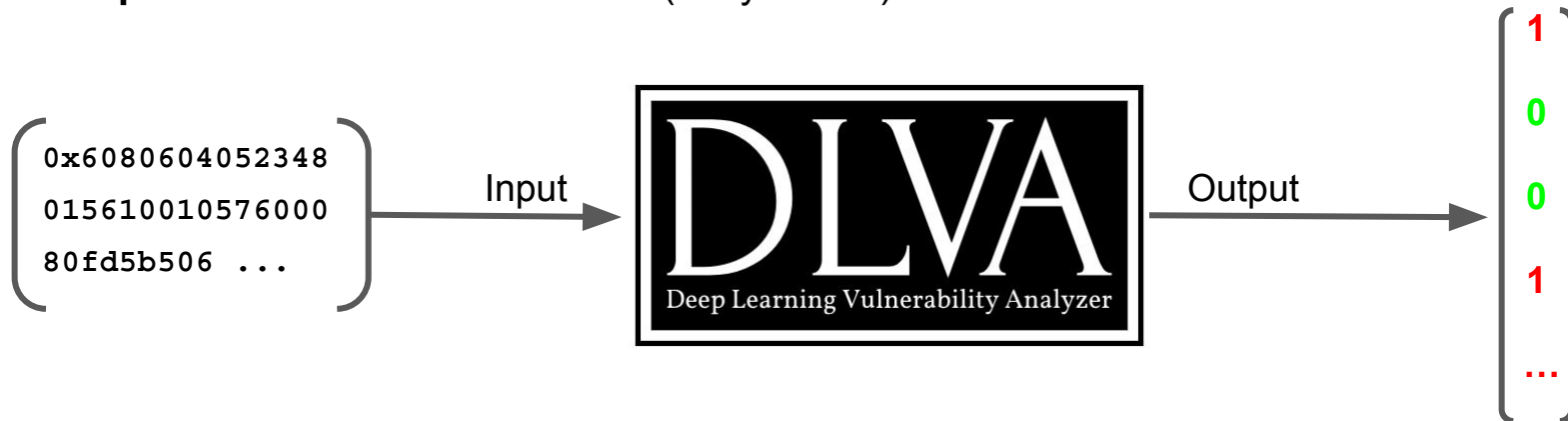
The 32nd USENIX Security Symposium ([USENIX Security 23](#)).

Contributions

- **DLVA: Deep Learning Vulnerability Analyzer**
 - Introduces a novel method for detecting **29** smart contract vulnerabilities in bytecode.
 - Achieves high accuracy with minimal false positives.
 - Rapid response times, faster (**5x-1,000x**) than state-of-the-art analyzers.
- **A Tool - DLVA**
 - Awarded three USENIX AEC badges (*Available, Functional, Reproduced*).
- **Evaluation**
 - Shows competitive performance against **eleven** state-of-the-art analyzers.
 - Scalable for extensive large-scale analyses.

DLVA Specification

- ❖ **Input:** Let **C** be a smart contract (in bytecode).



- ❖ **Output:** A 29-dimensional binary vector corresponding to the **29** distinct vulnerabilities. Each element in the vector indicates:
 - **1:** **C** is **vulnerable** to the corresponding vulnerability.
 - **0:** **C** is **secure** from the corresponding vulnerability.

Use Source Code Analyzer (Slither) to Train Bytecode Analyzer

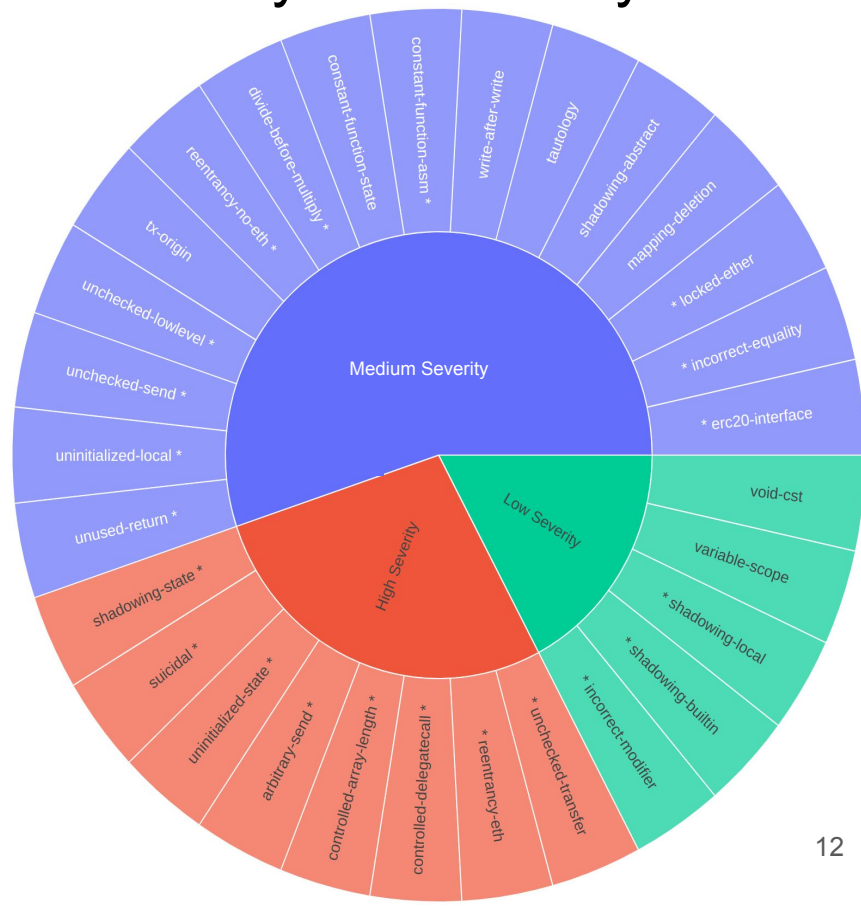
❖ Slither:

- Widely used and actively developed since 2019
- Now it detects **74** vulnerability types
- More accurate than competitors
- Requires source code availability

❖ Employ Slither to label two datasets:

- Large-length contract dataset (113K+ contracts):
29 vulnerabilities (200+ positive samples)
- Small-length contract dataset (7K+ contracts):
21 vulnerabilities* (30+ positive samples)

❖ Utilize these labeled datasets to train a bytecode analyzer (DLVA).



Smart contract representations

Solidity source code

```
contract Bank{  
    mapping(address => uint) balances;  
  
    function deposit() public payable {  
        uint amount = balances[msg.sender];  
        balances[msg.sender] = amount + msg.value; }  
  
    function withdraw() public {  
        if (msg.sender.call.value(balances[msg.sender])) {  
            balances[msg.sender] = 0; } }  
}
```



Compiler

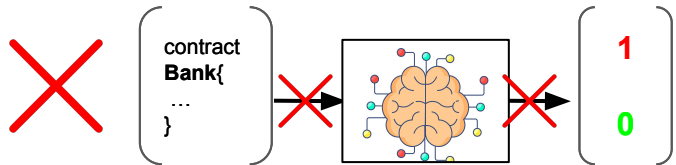
Bytecode

```
0x6080604052348015  
61001057600080fd5b  
506101608061002060  
00396000f3fe608060  
405234801561001057  
600080fd5b50600436  
1061002b5760003560  
e01c80633ccfd60b14  
610030575b600080fd  
5b ...
```

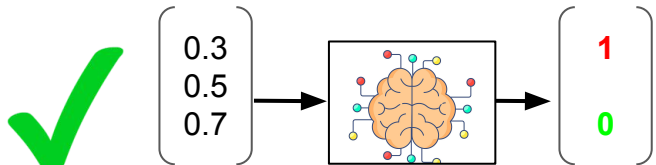
Disassembler

Opcodes

```
...  
  
JUMPDEST  
SWAP1  
SLOAD  
SWAP1  
PUSH2 0x0100  
EXP  
SWAP1  
DIV  
PUSH1 0x01  
...
```

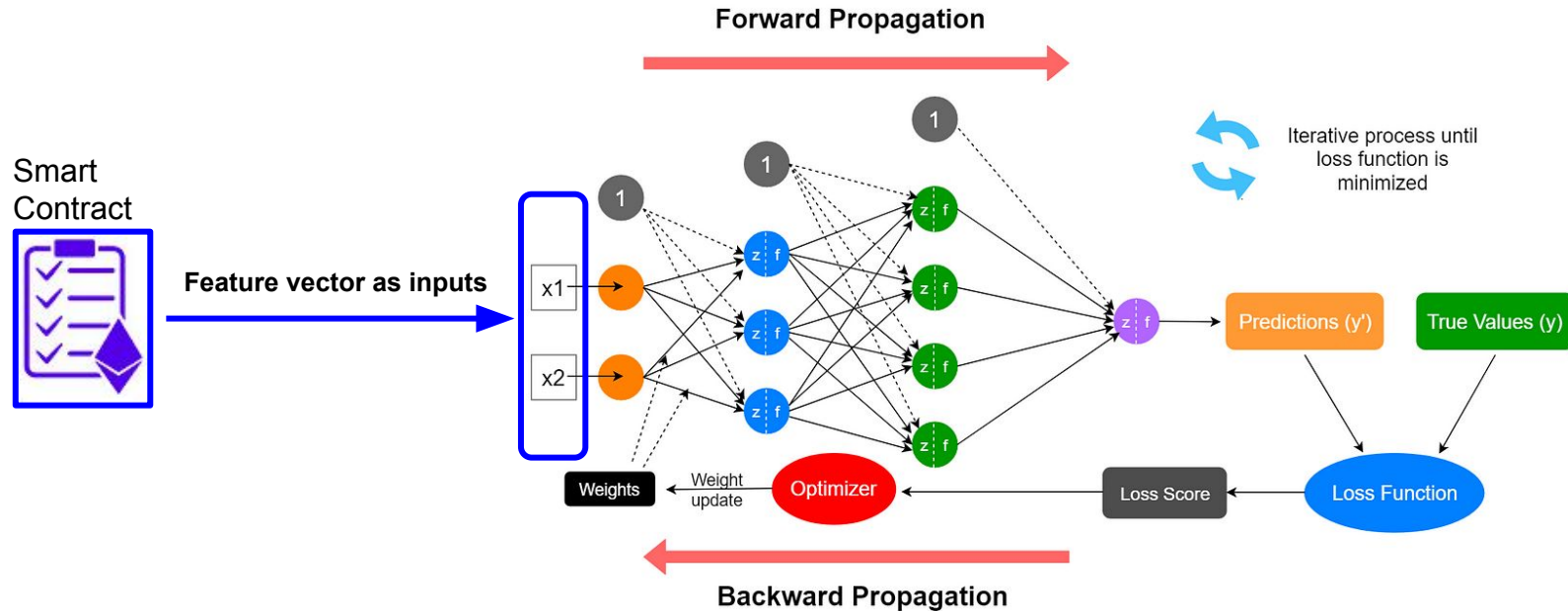


Neural networks can't interpret contract code



Neural networks require numerical feature vectors

Neural Network inputs



Primary Challenge:

Transforming smart contracts into **feature vectors** suitable for Neural Network inputs.

Feature Vector Extraction: Manual vs. Neural Networks

```
contract Bank{  
  deposit(){...}  
  withdraw(){...}  
}
```

Manual Feature Extraction

- + Used in many related work, and achieved good performance
- Predefined features
- Time consuming
- Not scalable in practice

EXTRACTED FEATURES FROM AST			
#	Feature Name	#	Feature Name
1	Lines of code (LOC)	10	Bytes
2	Contract definitions	11	Elementary type addresses
3	Function definitions	12	Modifier definitions
5	Binary operations	13	Placeholder statements
6	Function calls	14	Modifier invocation
7	Blocks	15	Approve function definitions
8	Expression statements	16	Constant values
9	Event definitions	17	Hexadecimal addresses

Source: Machine learning model for smart contracts security analysis (PST 2019).

```
0x608060405  
23480156100  
1057600080f  
d5b506 ...
```

Feature Learning using NN

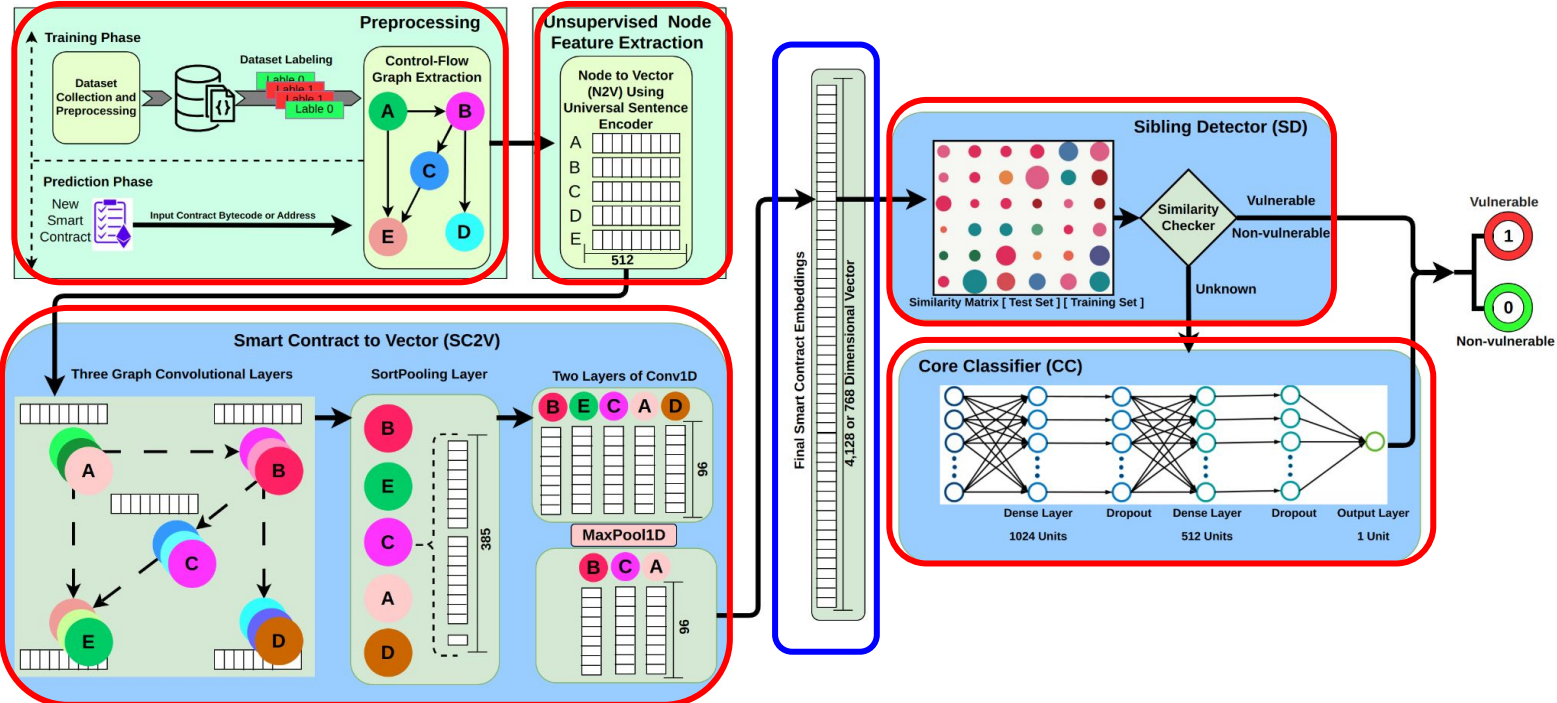
- + Automatic feature extraction during the learning process
- + Fast and scalable
- Requires many hyperparameter tuning experiments

[0.032677 0.027598 0.014528 0.021004 ... 0.107124 0.114875 0.121002]

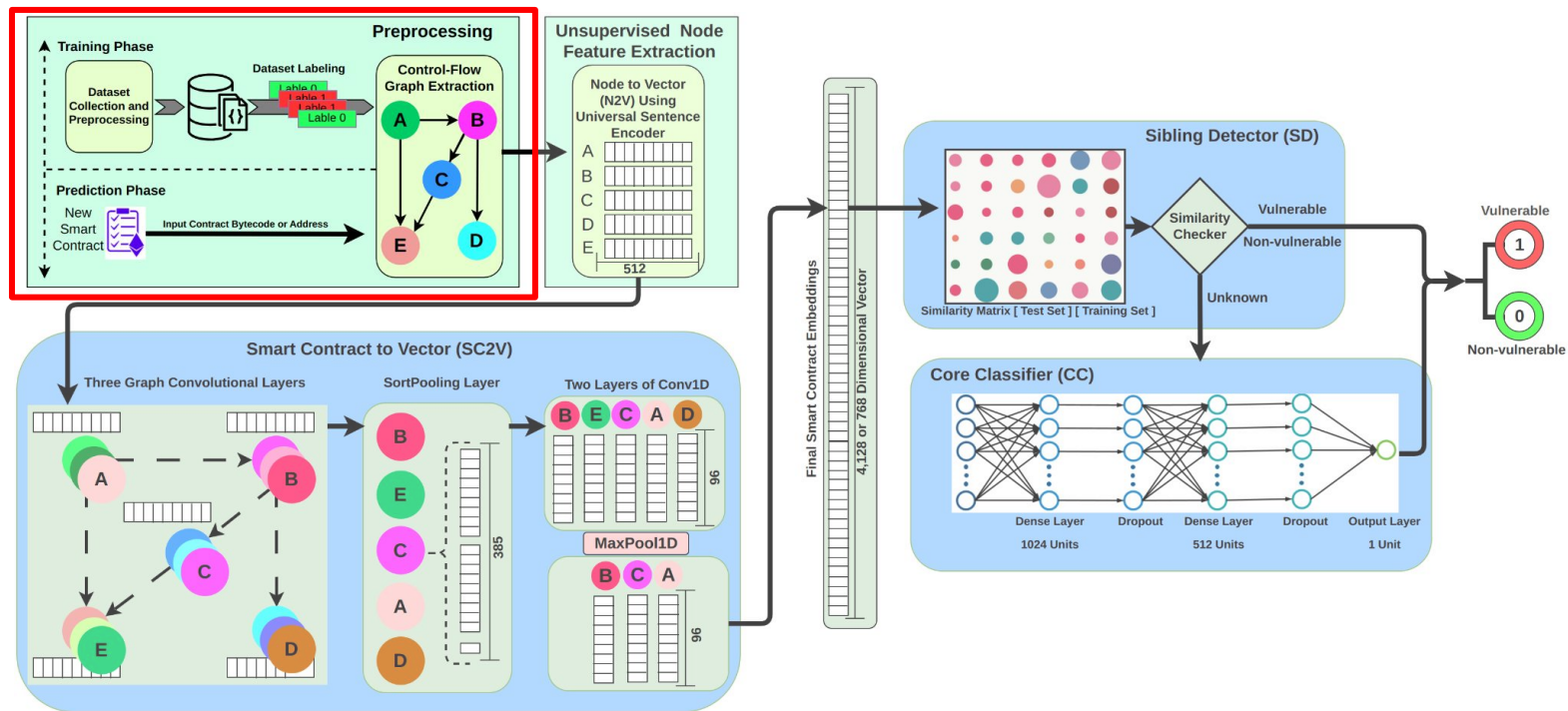
Source: Smart Learning to Find Dumb Contracts (USENIX Security 2023).

Architecture of DLVA

- 🔑: Use graph convolution NN to extract semantic structure
- 🔑: Neural networks learn from feature vectors to classify contracts

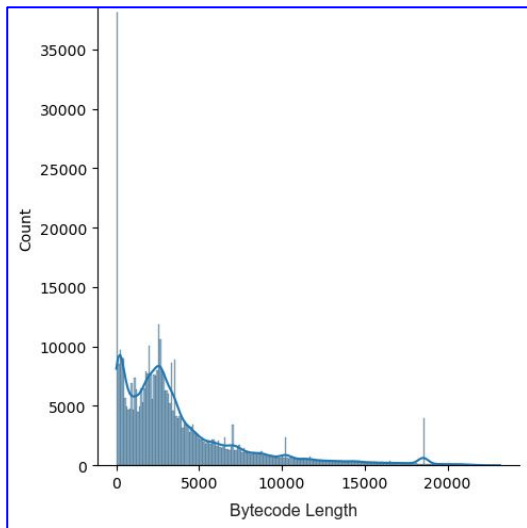


Preprocessing



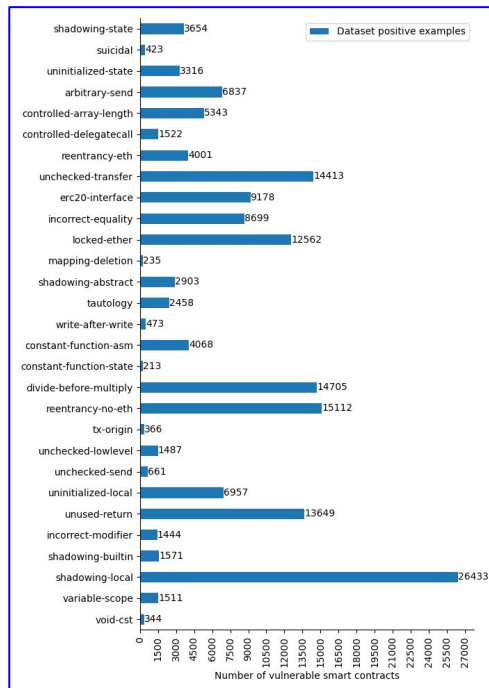
Preprocessing

1. Dataset Collection



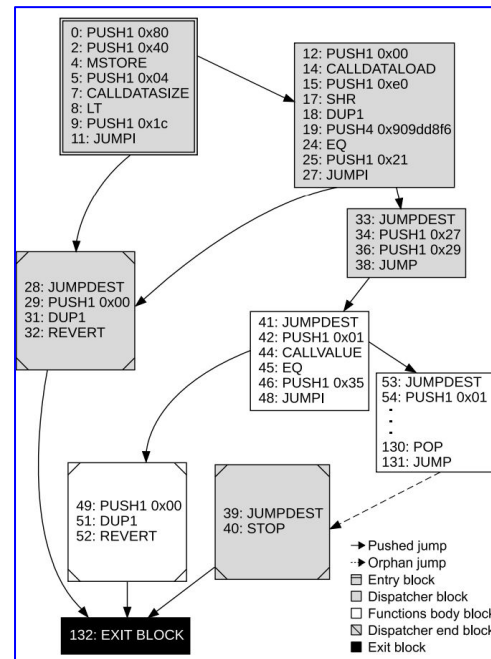
- **51,913,308** real-world contracts
- Remove redundant **99.3%**
- Only **368,438** are unique contracts

2. Dataset Labeling



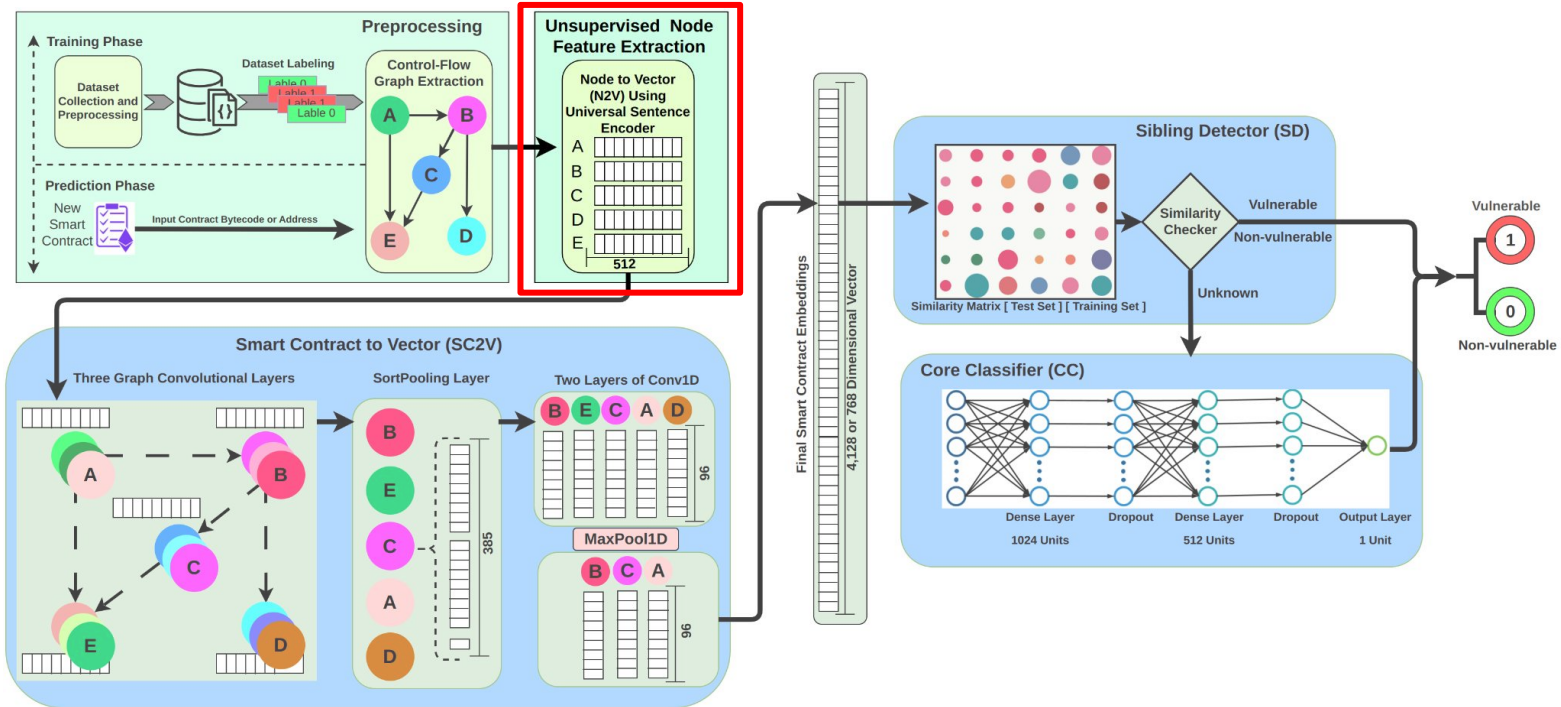
- Slither requires source code available
- Only $\frac{1}{3}$ of unique contracts are labeled

3. CFG extraction



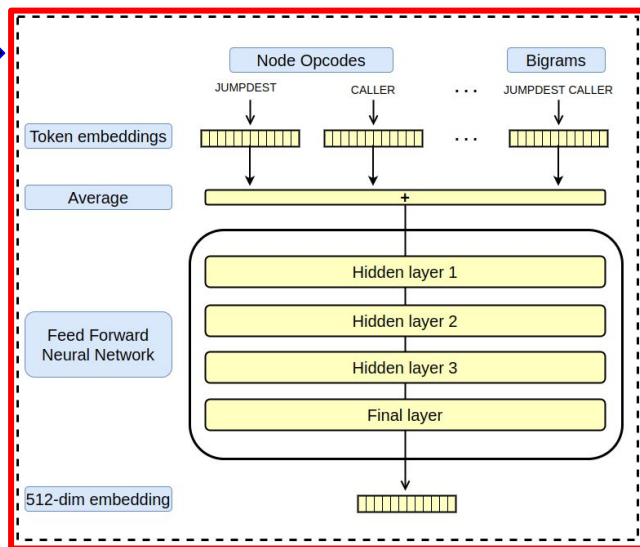
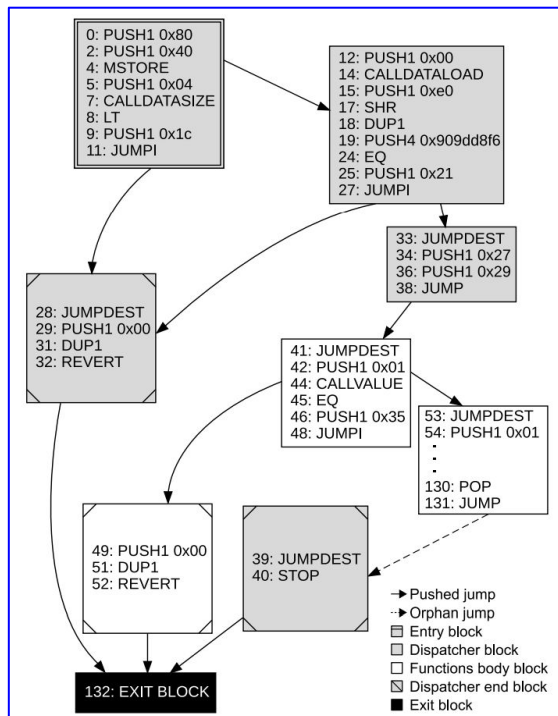
- CFG captures important semantics structures

Node to Vector (N2V)

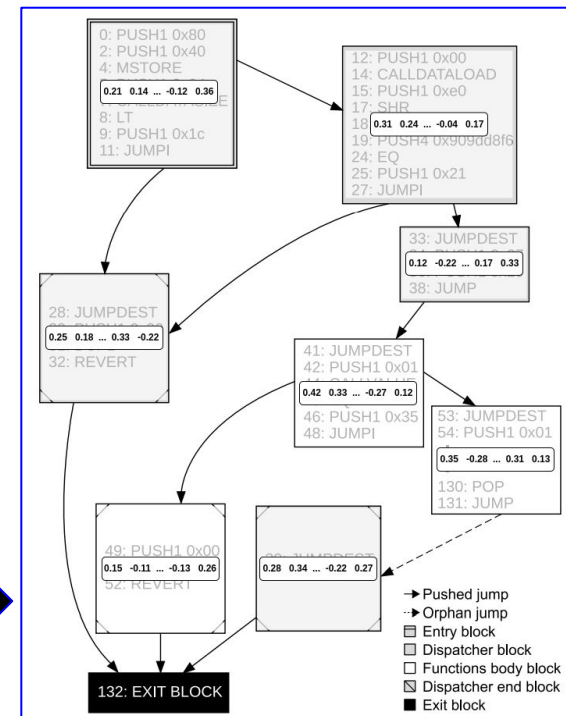


Node to Vector (N2V)

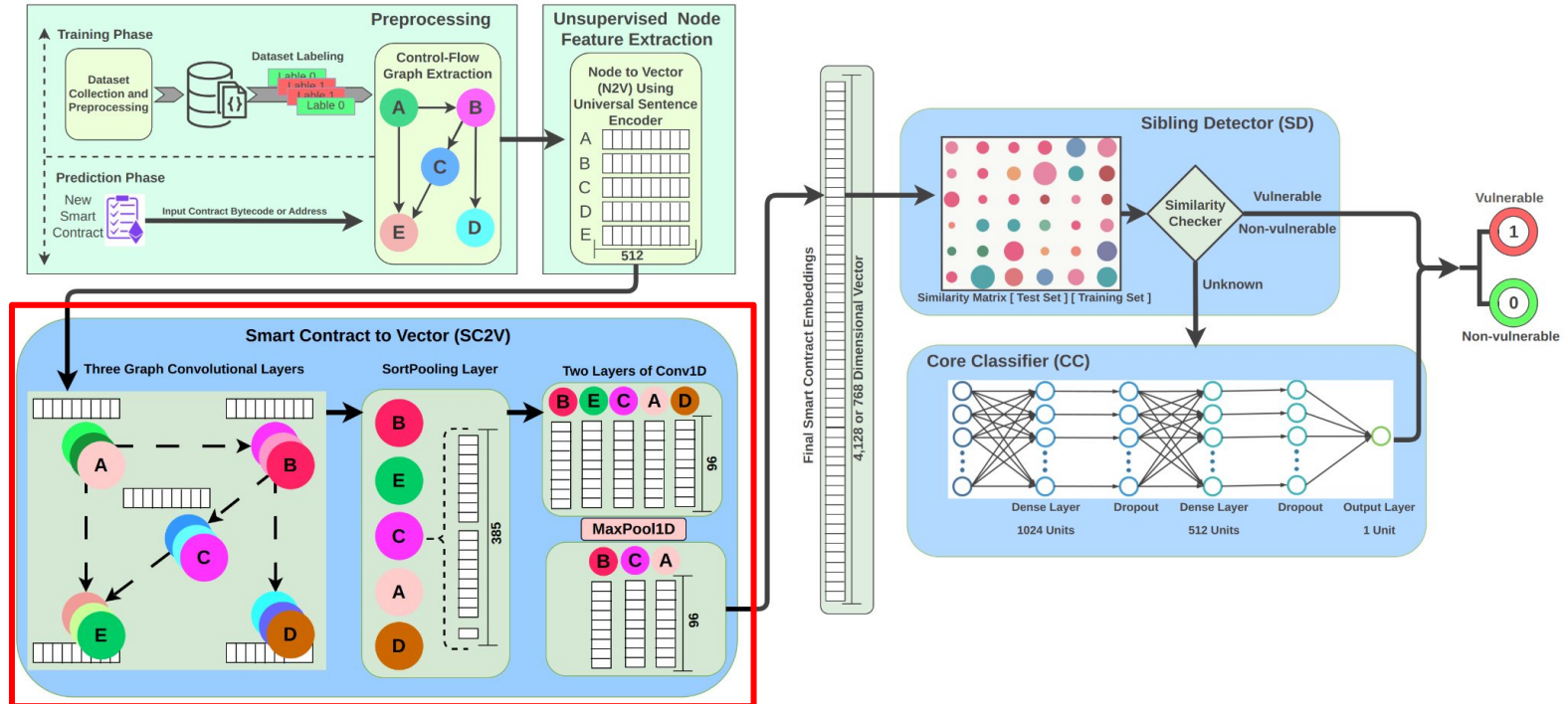
🔑: Similar basic blocks/nodes → Similar vectors in embedding space



Unsupervised Deep Averaging Network (DAN)
of the Universal Sentence Encoder

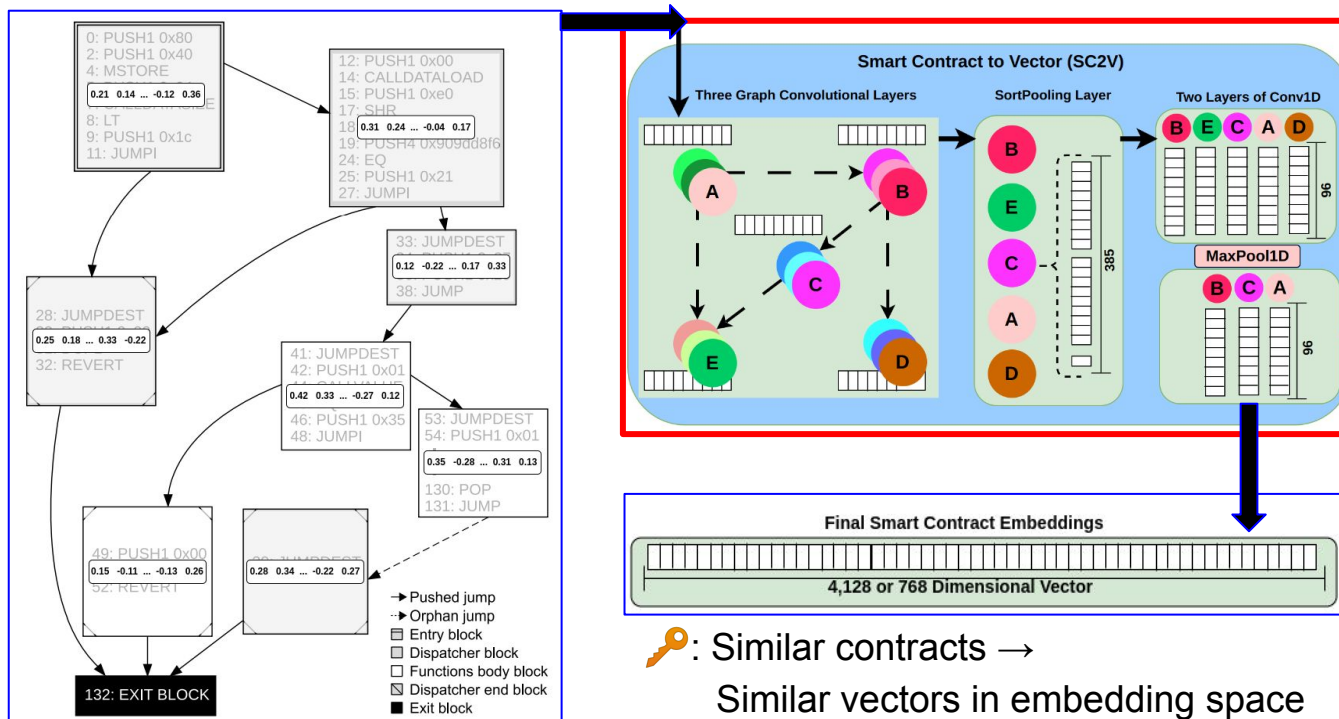


Smart Contract to Vector (SC2V)

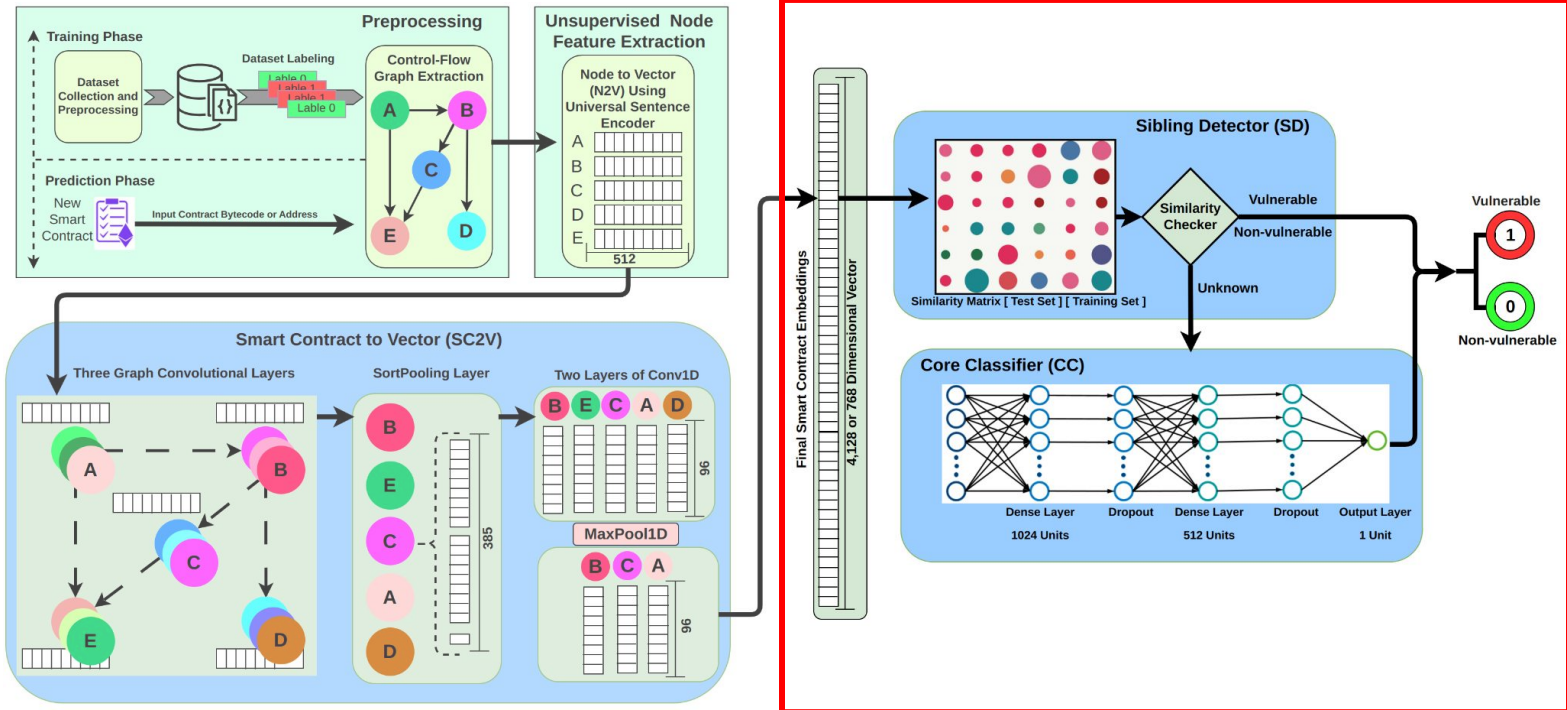


Smart Contract to Vector (SC2V)

- SC2V uses node summaries to make a vector summary of the entire CFG
- : Use graph convolution NN to extract semantic structure from CFG



Sibling Detector (SD) and Core Classifier (CC)

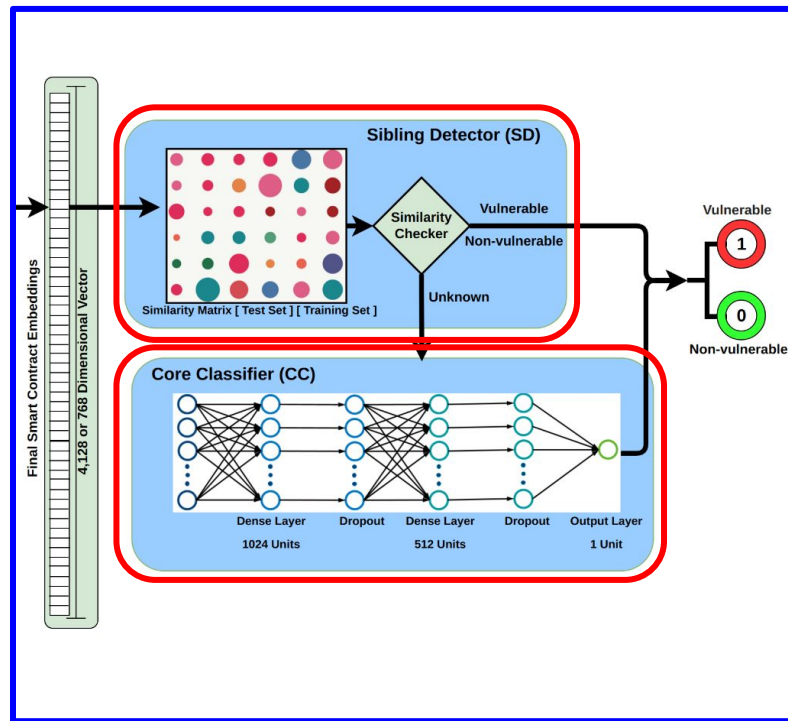


Sibling Detector (SD) and Core Classifier (CC)

- ★ SD tries to find a Euclidean-close neighbor to the target contract from the training data

$$d(\mathbf{p}, \mathbf{q}) = \sqrt{\sum_{i=1}^n (q_i - p_i)^2}$$

- ★ If no close neighbor, SD reports “**unknown**”
- ★ CC only called when SD reports unknown
- ★ CC uses FFN to label smart contracts regardless of vector distance.



Hyperparameter Tuning and Training Experiments

- ★ **Hyperparameter Search:** Conducted **972** hyperparameter tuning experiments on a **single** vulnerability type (**constant-function-asm**) exclusively detectable by Slither, and isn't the focus of the critical testing done later. This minimizes the risk of testing bias caused by overfitting.
- ★ **Picking the Winner:** Chose the top-performing hyperparameters for the final DLVA's design.
- ★ **Scaling Up:** The winning design, proven on **one** vulnerability type, successfully scales to **28** additional vulnerability types.
- ★ **Generic Design:** DLVA's adaptable design ensures readiness for even **more** vulnerability types.
- ★ We had **972**(Hyperparameter tuning) + **28**(Large-length contracts) + **21**(Small-length contracts) = **1,021** models to train. Total wall-clock training time was approximately **four days**.

Evaluation: Datasets used to benchmark DLVA

- ★ There's a lack of reliable benchmarks for assessing smart contract vulnerability detection.
- ★ Developing robust benchmarks is a critical need for the blockchain security community.

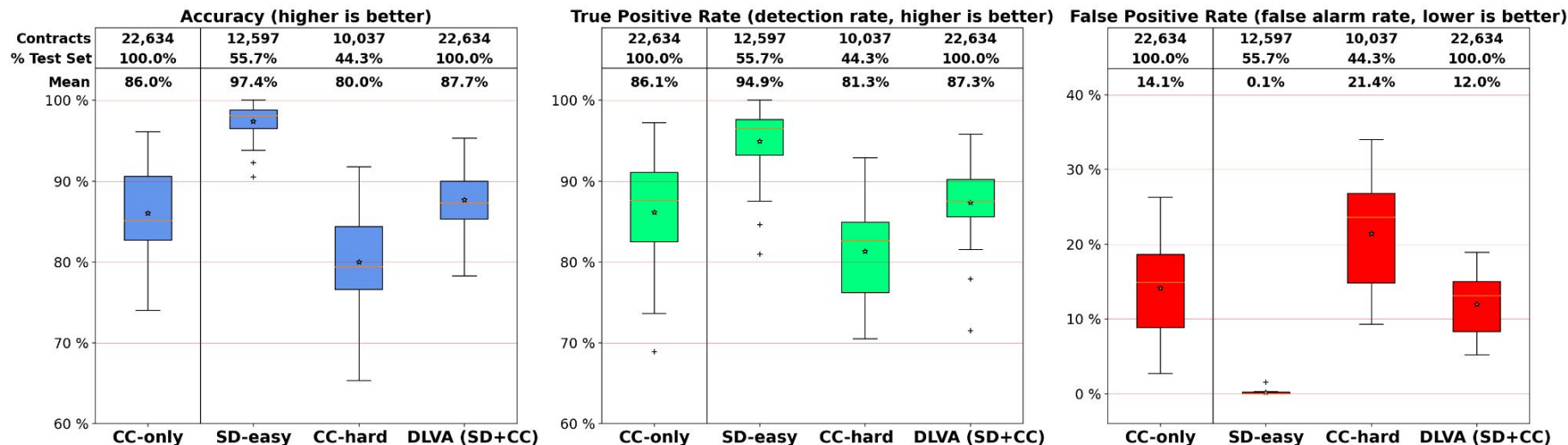
Dataset	No. of contracts	No. of vulnerabilities	Contract size	Ground Truth
<i>EthereumSC_{large}</i> [3]	22,634	29	Large	Slither
<i>EthereumSC_{small}</i> [4]	1,381	21	Small	Slither
<i>Elysium_{benchmark}</i> [2]	900 (57)	2	Small	Peer-reviewed
<i>Reentrancy_{benchmark}</i> [6]	473 (472)	1	Small	GP: Manual and GN: 2 analyzers
<i>SolidiFI_{benchmark}</i> [8]	444	4	Large	GP: Bug injection and GN: 5 analyzers

🔑: All datasets are disjoint from DLVA's training sets

🔑: All datasets are publicly available

Results on Slither ground truth prediction

★ EthereumSCLarge Test Set



SD & CC boosted DLVA: **+1.7%** accuracy, **+1.2%** true positives, **-2.1%** false positives. ⚡

★ EthereumSCsmall Test Set

DLVA impresses with **97.6%** accuracy, effectively detecting positives (**95.4%** TPR) while keeping false positives minimal (**2.3%** FPR).

Competitors for benchmarking

★ DLVA is benchmarked vs. **eleven** State-of-the-Art tools

★ Using **three** benchmarks:

- *Elysium*_{benchmark}
- *Reentrancy*_{benchmark}
- *SolidiFI*_{benchmark}

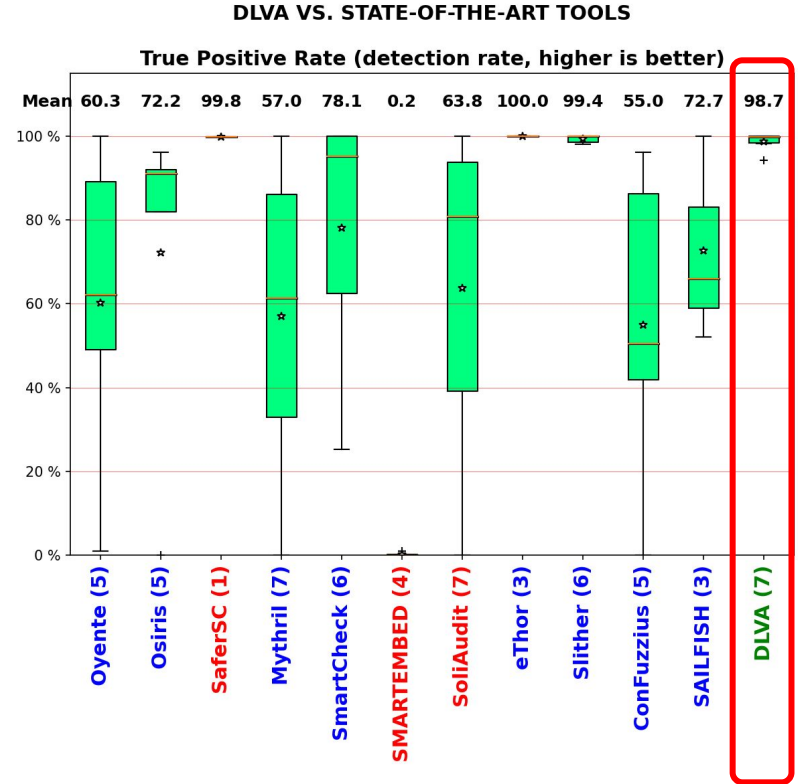
★ DLVA outperforms **11** top tools across **3** benchmarks.

Analyzer	Input	Method	Vulnerabilities	Year	Citations
Oyente 0.2.7	Source/Bytecode	SA/DA	4	2017	2,219
Osiris	Source/Bytecode	SA/DA	5	2018	287
SaferSC	Bytecode	ML/DL	1*	2018	97
Mythril 0.21.20	Source/Bytecode	SA/DA	13	2019	153
SmartCheck 2.0	Source code	SA/DA	43	2019	636
SMARTMBED	Bytecode	ML/DL	10	2019	97
SoliAudit	Source code	ML/DL	13	2019	98
eThor	Bytecode	SA/DA	1	2020	107
Slither 0.8.0	Source code	SA/DA	74	2021	411
ConFuzzius	Source code	SA/DA	10	2022	46
SAILFISH	Source code	SA/DA	2	2022	46
DLVA	Bytecode	ML/DL	29	2023	NA

DLVA is highly sensitive

	Predicted Positive	Predicted Negative
Positive	True Positive ✓	False Negative ✗
Negative	False Positive ✗	True Negative ✓

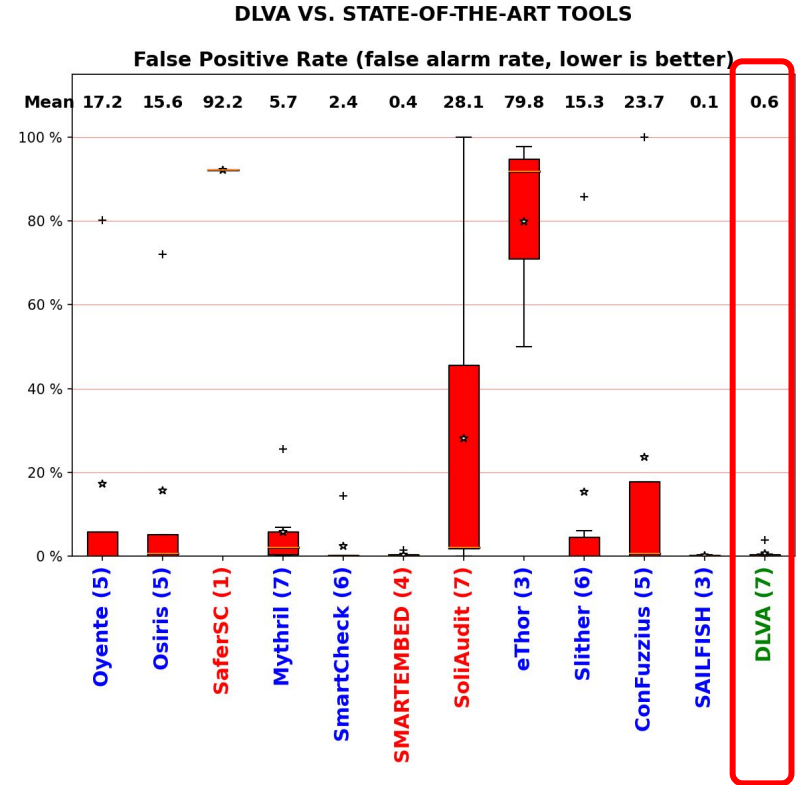
$$\text{True Positive Rate (TPR)} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$



DLVA is highly selective

	Predicted Positive	Predicted Negative
Positive	True Positive ✓	False Negative ✗
Negative	False Positive ✗	True Negative ✓

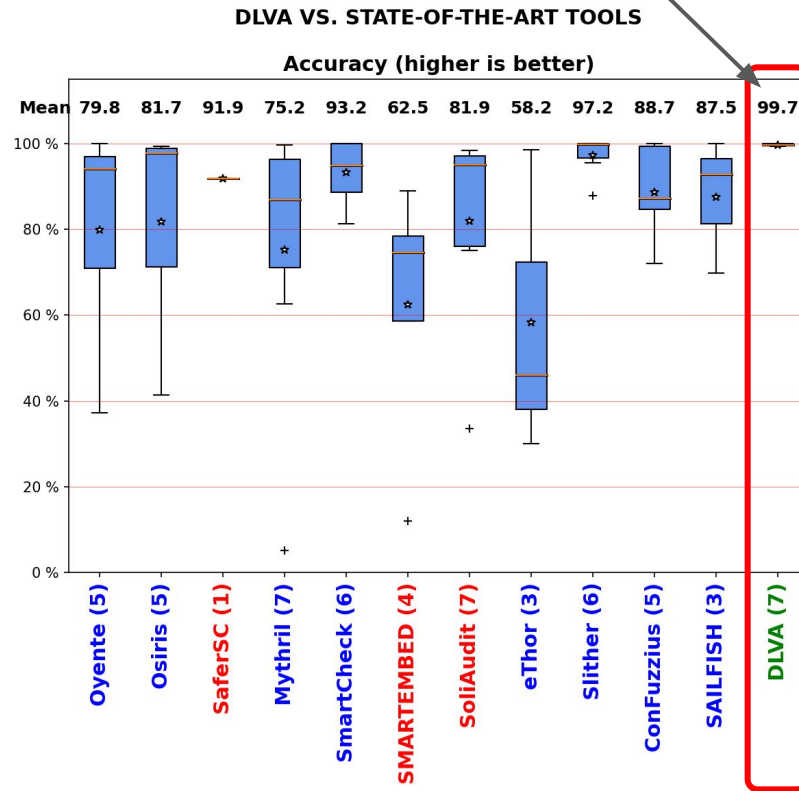
$$\text{False Positive Rate (FPR)} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$



DLVA is highly accurate

	Predicted Positive	Predicted Negative
Positive	True Positive ✓	False Negative ✗
Negative	False Positive ✗	True Negative ✓

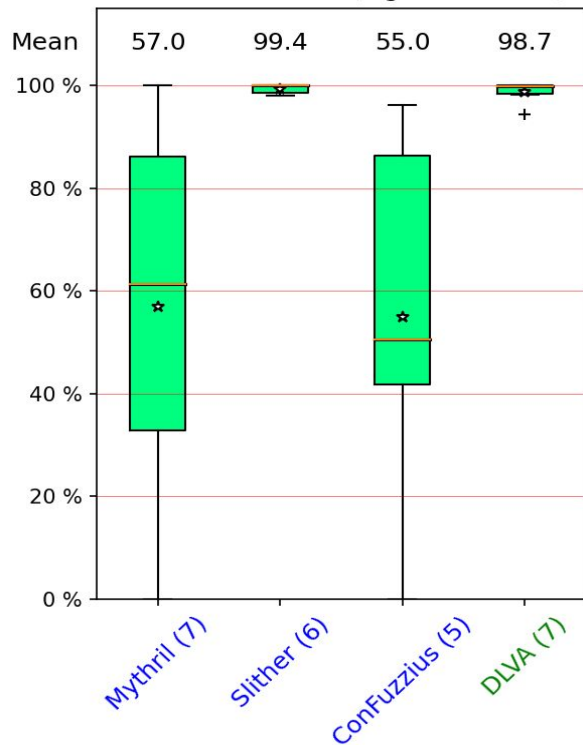
$$\text{Accuracy} = \frac{(\text{TP} + \text{TN})}{(\text{TP} + \text{FP} + \text{TN} + \text{FN})}$$



DLVA compared to widely used Tools

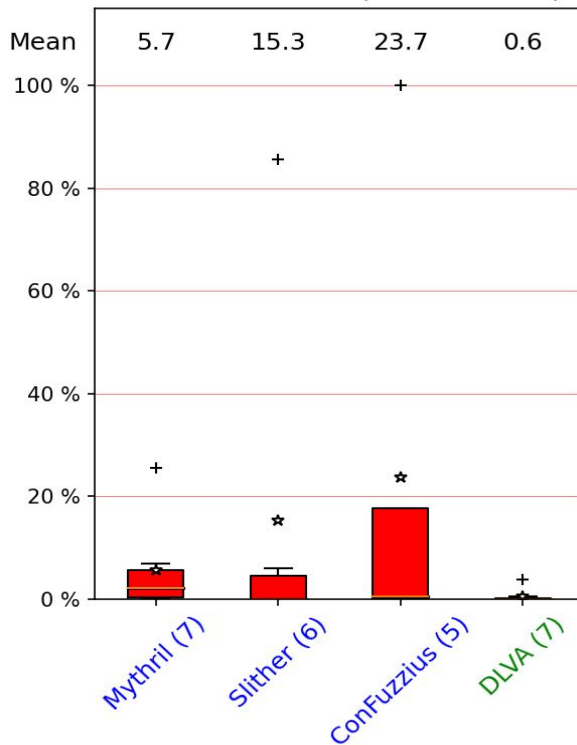
DLVA VS. STATE-OF-THE-ART TOOLS

True Positive Rate (higher is better)



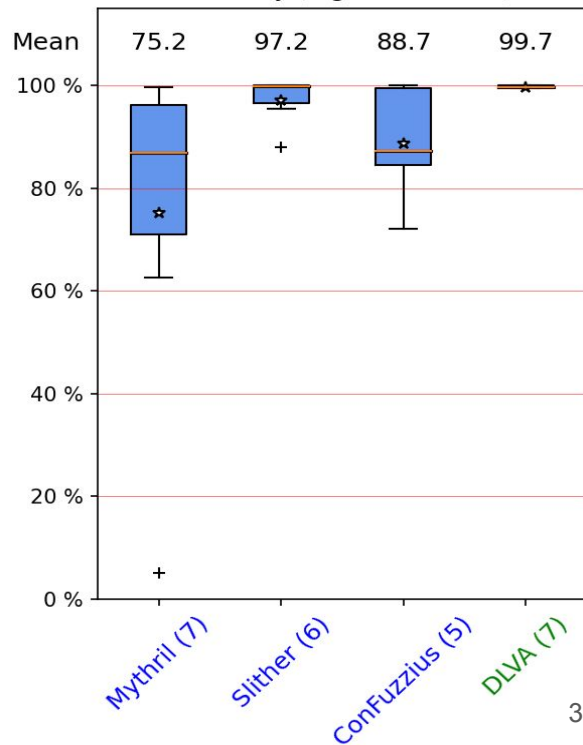
DLVA VS. STATE-OF-THE-ART TOOLS

False Positive Rate (lower is better)



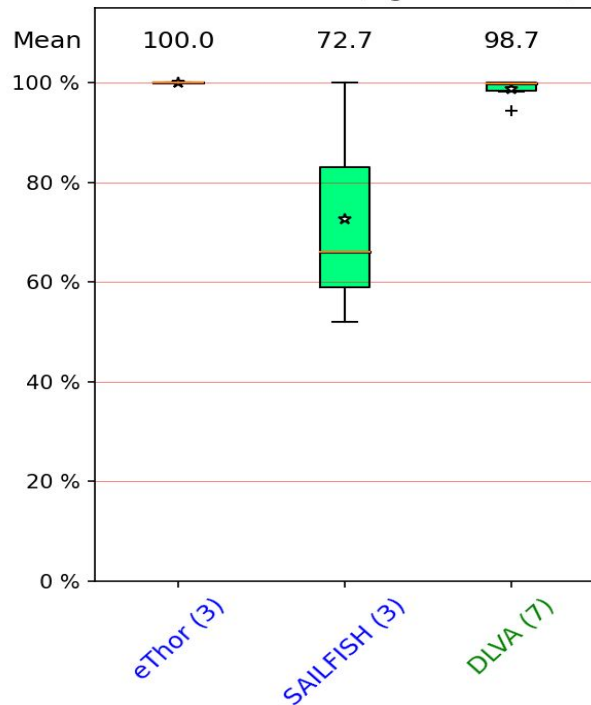
DLVA VS. STATE-OF-THE-ART TOOLS

Accuracy (higher is better)

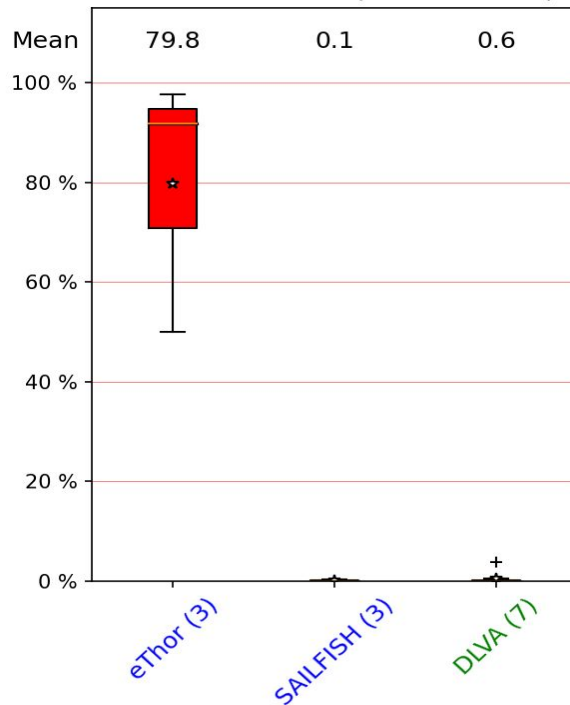


DLVA compared to the best TPR/FPR Tools

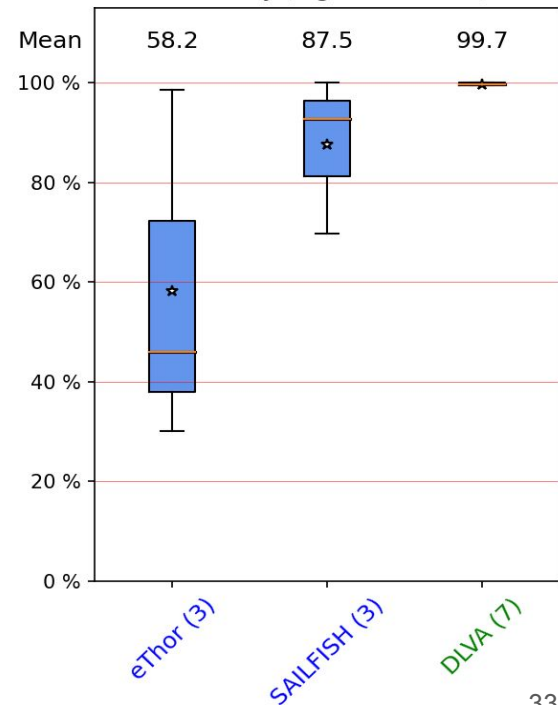
DLVA VS. STATE-OF-THE-ART TOOLS
True Positive Rate (higher is better)



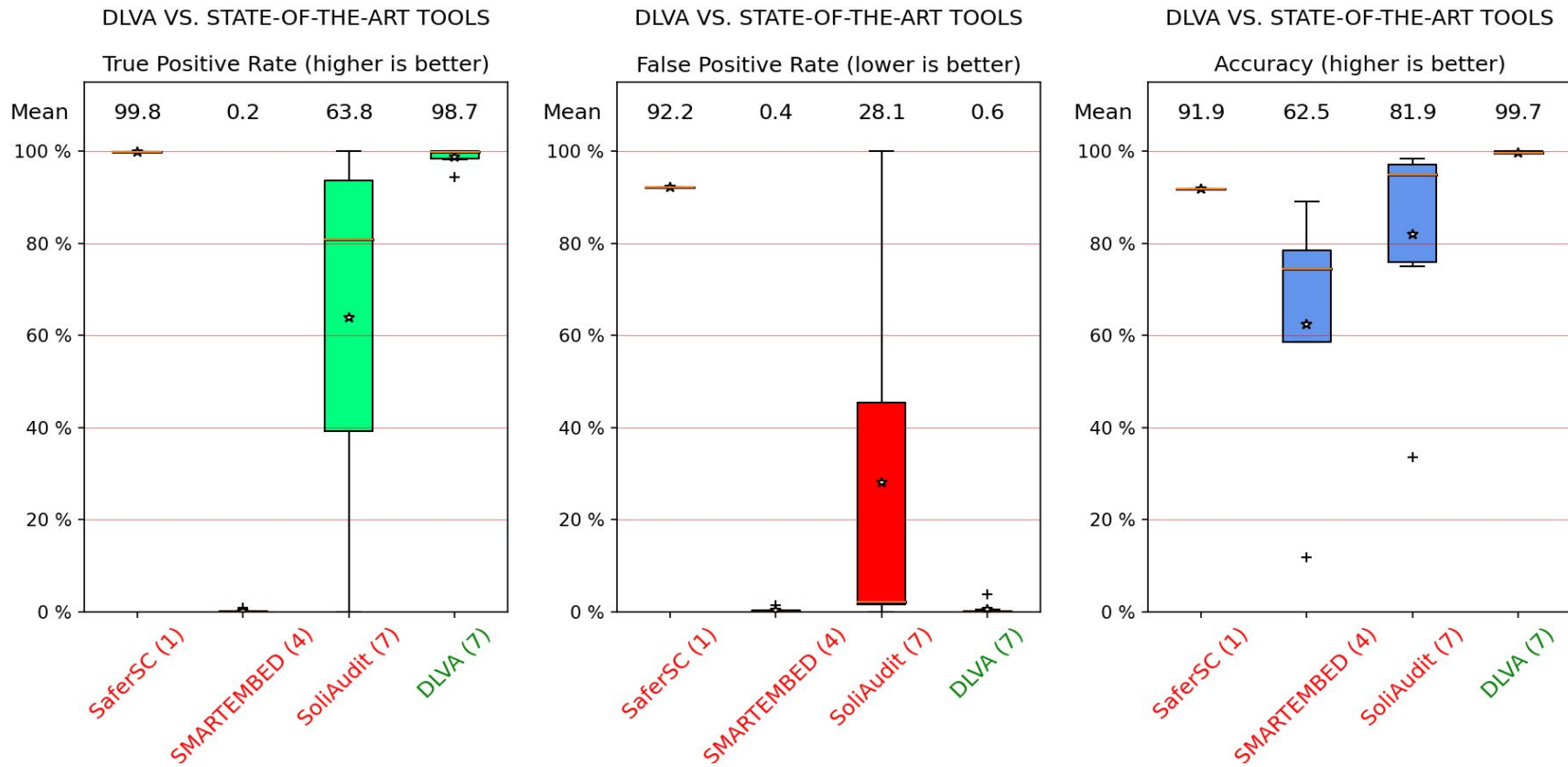
DLVA VS. STATE-OF-THE-ART TOOLS
False Positive Rate (lower is better)



DLVA VS. STATE-OF-THE-ART TOOLS
Accuracy (higher is better)



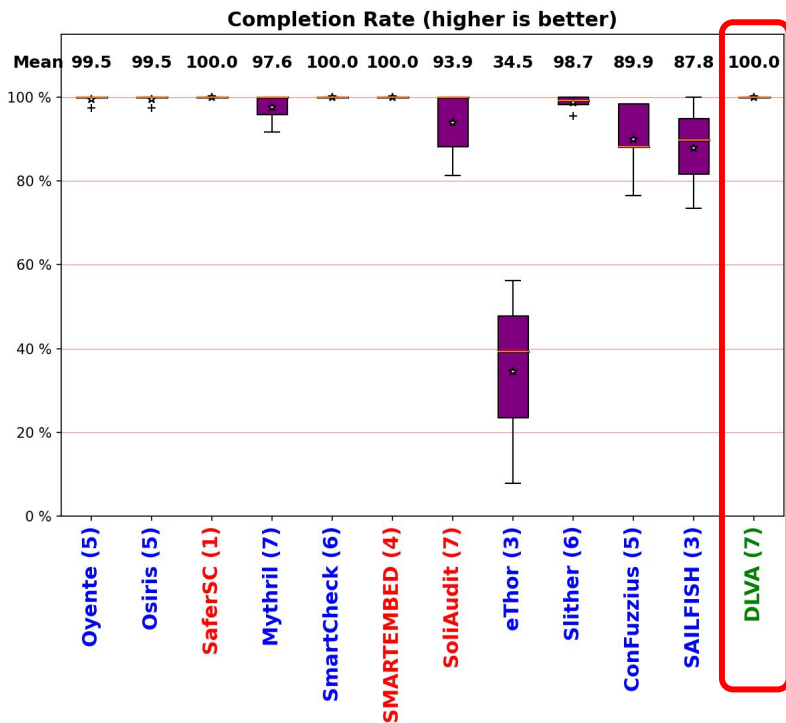
DLVA compared to available ML/DL Tools



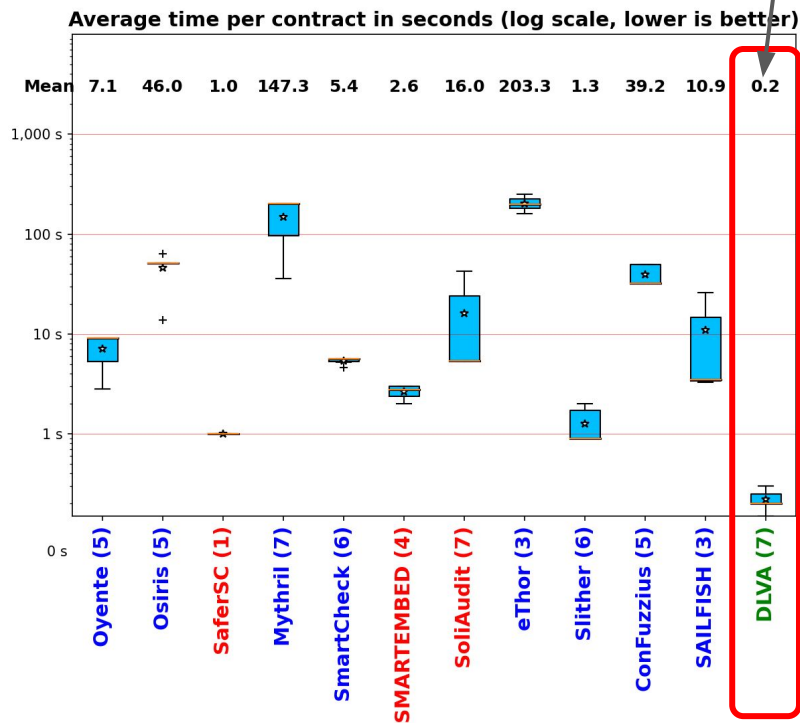
DLVA almost always answers... and very quickly

DLVA is **5x – 1,000x** faster.

DLVA VS. STATE-OF-THE-ART TOOLS



DLVA VS. STATE-OF-THE-ART TOOLS



Key Takeaways

- **Novel Approach: Deep Learning Vulnerability Aalyzer (DLVA)**
 - Detects a wide range of distinct vulnerability types.
 - Analyzes smart contract bytecode directly, eliminating the need for source code.
 - Accelerates the analysis process, improving detection performance metrics.
- **Evaluation**
 - Measures DLVA's performance in predicting Slither's labels.
 - Demonstrates competitive performance compared to **eleven** state-of-the-art analyzers across **three** independent benchmarks.

SCooLS : Smart Contract Learning (Semi-supervised)

Proposal 2: SCooLS

[IEEE BCCA 23]

Exploit Detection

Lack of labels

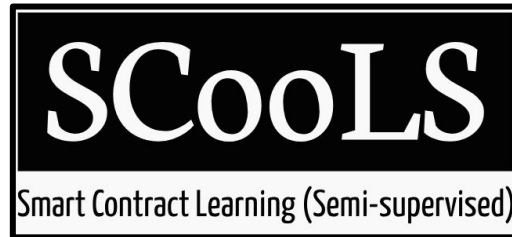
Localization

Speed

Accuracy

Availability

1 Vulnerability & Auto-exploit



Tamer Abdelaziz and Aquinas Hobor. “Schooling to Exploit Foolish Contracts”.

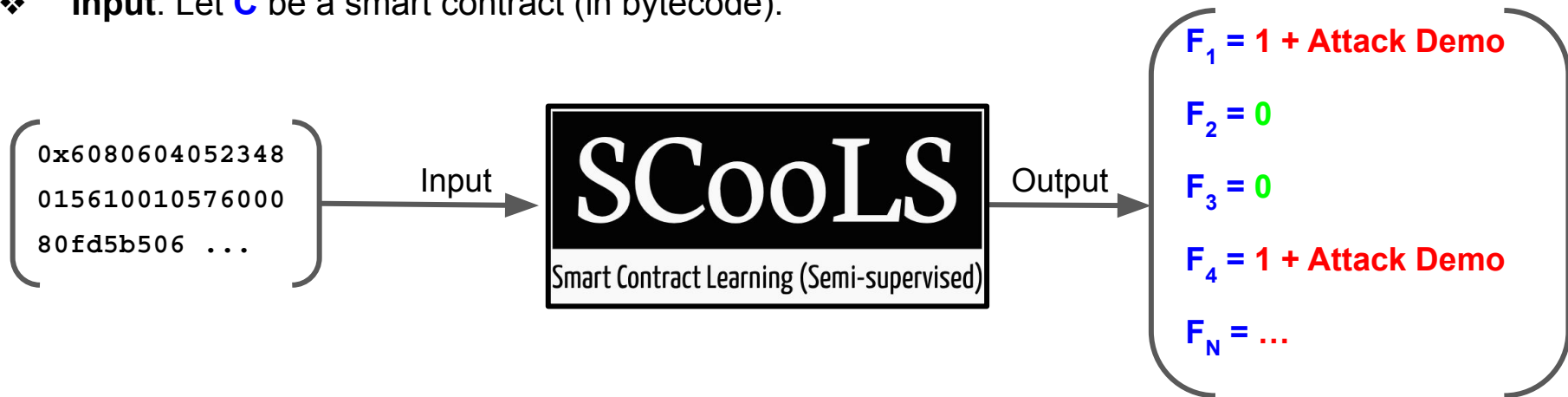
The Fifth International Conference on Blockchain Computing and Applications (IEEE BCCA 23).

Contributions

- **SCoolS: Semi-supervised Expolit Analyzer**
 - Addresses limited labeled data for precise detection of specific **exploitable functions**.
 - Introduces a voting committee for generating high-confidence labels during training cycles.
 - Generates attack demonstrations for developers, facilitating contract exploitability testing.
 - High accuracy with minimal false positives, and faster than competitors.
- **A Tool - SCoolS**
 - Offers a publicly available implementation tool (SCoolS).
- **Evaluation**
 - Demonstrates competitive performance against **three** state-of-the-art analyzers.
 - Scalable for extensive large-scale analyses.

SCoolS Specification

- ❖ **Input:** Let **C** be a smart contract (in bytecode).



- ❖ **Output:** N-dimensional binary vector, where **N** is the number of functions in the contract:
 - **1:** Function F_i in **C** is **exploitable** for reentrancy, with **attack demonstration**.
 - **0:** Function F_i in **C** is **secure** from reentrancy.

Key Elements of SCoolS

★ Semi-supervised Learning cycles:

- Initial models trained on a small **hand-labeled** dataset.
- Use these initial models to label a large **unlabeled** dataset.
- Train the next generation of models using high-confidence labels.

★ Graph Neural Networks (GNNs):

- Analyzes smart contract bytecode at **function-level**.

★ Localization & Auto-exploit:

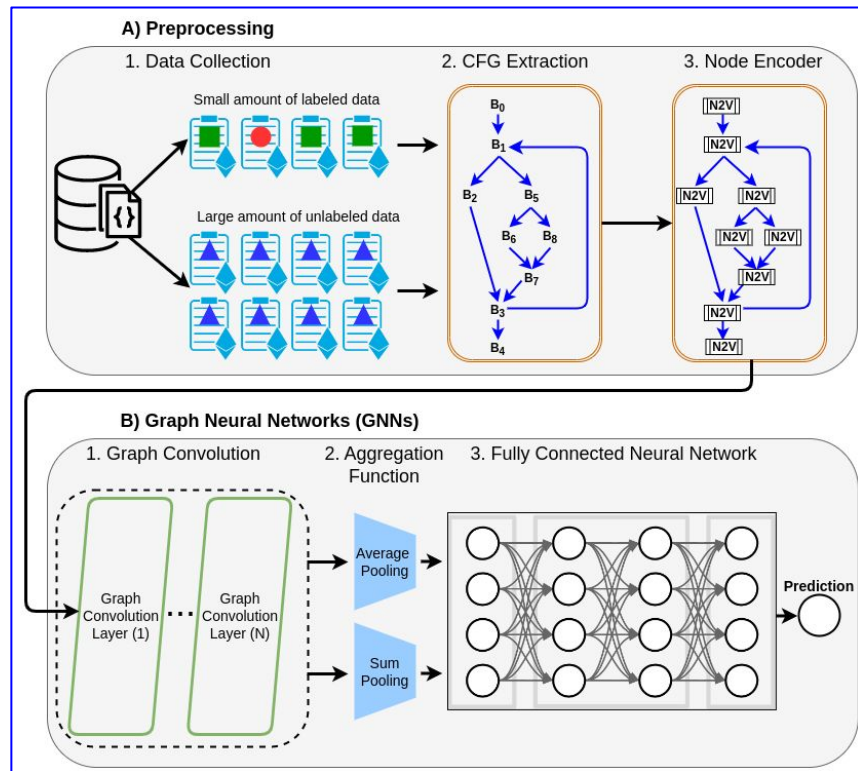
- Pinpoints the exact function responsible for reentrancy.
- Shows for the developers how attackers exploit reentrancy vulnerabilities.

Dataset Overview

- ★ **Focus on Function-Level Analysis:** DLVA benchmarks were not used due to their contract-level labeling, and employ differing definitions of vulnerability and exploitability.
- ★ **ReentrancyBook Dataset:**
 - 502 unique and hand-labeled functions by our team.
 - Pinpoints reentrancy vulnerabilities directly at the function-level.
 - Provides reliable ground truth for model training and testing.
- ★ **BigBook Dataset:**
 - Massive scale: 17M+ contracts, 553K+ unique functions (after deduplication).
 - Unlabeled dataset.
 - Powers the semi-supervised learning technique to expand knowledge.

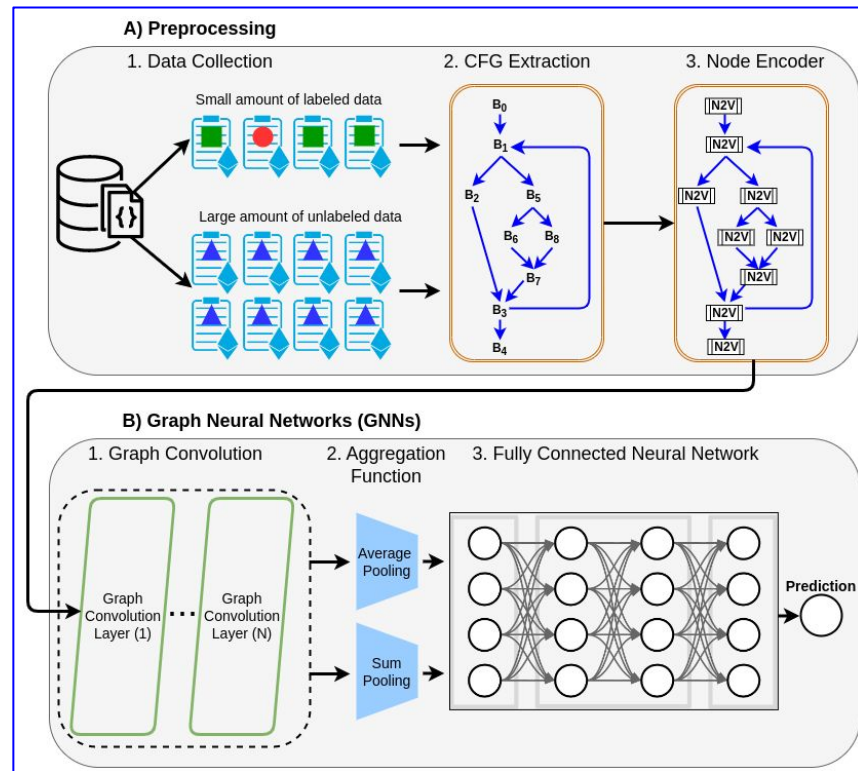
SCoolS Design (A: Data Preprocessing)

- ★ Adopts a similar approach to DLVA.
- ★ However, **hand-labeled** a small portion of data, leaving a larger portion **unlabeled**.
- ★ Extracts **function-level** CFG from bytecode.
- ★ Employs a powerful **Transformer Architecture** for node embeddings (while DLVA uses DAN).



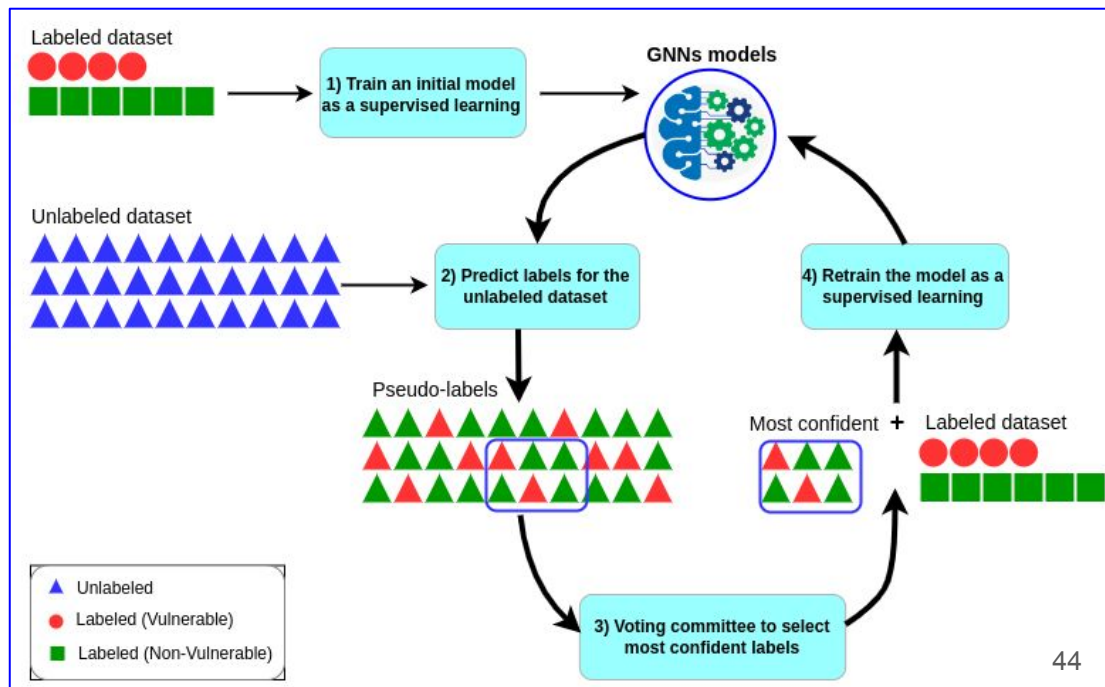
SCoolS Design (B: Graph Neural Networks)

- ★ Utilizes simpler **aggregation** techniques compared to DLVA, optimized for smaller CFGs.
- ★ Employs a **voting committee** of models instead of extensive hyperparameter tuning.
- ★ Constructs models with a series of graph convolutional layers, aggregation functions, and a fully connected feed-forward network.
- ★ Generates **120 unique models** from the hyperparameters, forming a diverse committee for robust classification.



SCoolS Design (C: Semi-Supervised Self-Training)

- ★ Begins with **hand-labeled** ReentrancyStudyBook to train the initial **120** models.
- ★ Employs the **120** models to evaluate the **unlabeled** BigBook.
- ★ Implements a **voting committee** to select most confident labels (**80+** votes).
- ★ Retrains models for the next iteration.
- ★ Repeats the training cycle until a majority of the BigBook are labeled.



Auto-Exploit Generator Design

★ We implement a simple yet effective exploit generator (combination of DL and Program Synthesis).

1. The victim contract TheBank

```
1 // SPDX - License - Identifier : MIT
2 pragma solidity ^0.8.17;
3 contract TheBank {
4     mapping(address => uint) theBalances;
5     function deposit() public payable {
6         theBalances[msg.sender] += msg.value;
7     }
8     function withdrawal() public {
9         uint bal = theBalances[msg.sender];
10        require ( bal > 0 );
11        (bool success, ) = msg.sender.call{value: bal}("");
12        require(success, "transaction failed");
13        theBalances[msg.sender] = 0;
14    }
15 }
```

SCoolS classified withdrawal() as exploitable

TheBank
ABI

```
[
  {
    "inputs": [],
    "name": "deposit",
    "outputs": [],
    "stateMutability": "payable",
    "type": "function"
  },
  {
    "inputs": [],
    "name": "withdrawal",
    "outputs": [],
    "stateMutability": "nonpayable",
    "type": "function"
  }
]
```

2. Attack template

```
1 interface Itarget{
2     /* signature of the payable function P */
3     /* signature of the vulnerable function V */
4 }
5 contract TheAttacker{
6     ...
7     function attack_step1() external payable {
8         /* call the payable function P */
9     }
10    function attack_step2() external {
11        /* call the vulnerable function V */
12    }
13    receive() external payable {
14        if ( has_funds ) {
15            /* call the vulnerable function V */
16        }
17    }
18    function steal() public payable{
19        attacker.transfer(address(this).balance);
20    }
21 }
```

3. The generator creates TheAttacker contract

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity >=0.1.0 <0.9.0;
3 interface Itarget{
4     function deposit() external payable;
5     function withdrawal() external;
6 }
7 contract TheAttacker{
8     Itarget public theBank;
9     address payable public attacker;
10    uint256 public amount = 1 ether;
11    constructor(address _theBankAddress, address payable _attackerAddr) {
12        theBank = Itarget(_theBankAddress);
13        attacker = _attackerAddr;
14    }
15    function attack_step1() external payable {
16        theBank.deposit{value: msg.value}();
17    }
18    function attack_step2() external {
19        theBank.withdrawal();
20    }
21    receive() external payable {
22        if (address(theBank).balance >= amount) {
23            theBank.withdrawal();
24        }
25    }
26    function steal() public payable{
27        attacker.transfer(address(this).balance);
28    }
29 }
```

Auto-Exploit Generator (Testing the exploit)

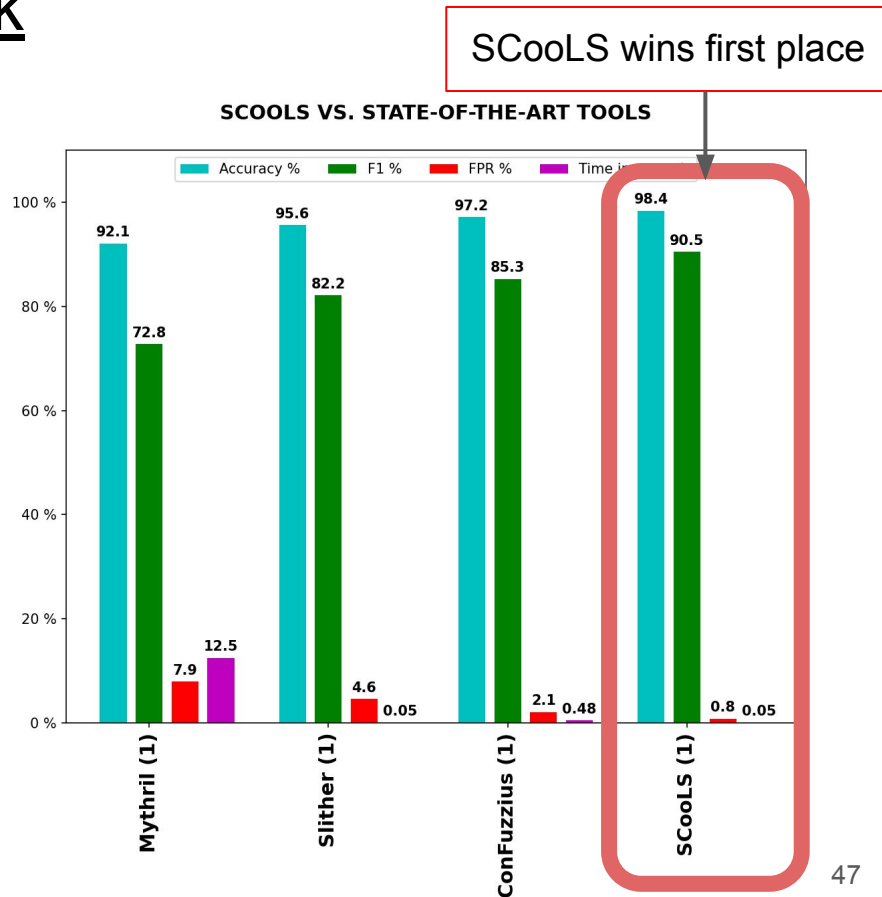
- ★ **Identification:** SCoolS's detector pinpoints exploitable functions within a victim contract.
- ★ **Attacker Contract Creation:** For each exploitable function, SCoolS's auto-exploit generator designs **N** attacker contracts corresponding to **N** payable functions.
- ★ **Local Blockchain Deployment:** These attacker contracts are deployed to a controlled local blockchain environment for safe and ethical testing.
- ★ **Impact Assessment:** The attacker's balance is monitored before and after each attack.
- ★ **Empower Developers:** SCoolS provides concrete evidence of financial risks, enabling developers to prioritize and address vulnerabilities effectively.

```
Attacker Balance Before Attack : 1808.65 Ether(s)
Attacker Balance After Attack   : 1810.64 Ether(s)
This exploitation net profit is: 1.99 Ether(s)
The detected reentrancy can be exploited.
```

Results on ReentrancyTestBook

- ★ Accuracy Level: **98.4%**
- ★ F1 Score: **90.5%**
- ★ Low False Positive Rate: **0.8%**
- ★ SCoolS' Speed in Analyzing a Typical Function: **0.05** seconds

Tool	TP	FP	TN	FN	ACC	F1	FPR	Time
SCoolS	9	2	239	2	98.4	90.5	0.8	0.05
ConFuzzius [45]	9	5	236	2	97.2	85.3	2.1	0.48
Slither [12]	11	11	230	0	95.6	82.2	4.6	0.05
Mythril [31]	10	19	222	1	92.1	72.8	7.9	12.5



Exploit Generation Results

★ Success Rate in Stealing Ether: **76.9%** of True Positives

THE EXPLOIT GENERATOR.

Address	Vulnerable function	Function duplicates	Exploit Gen
0x65e5909d665cb ...	CashOut(uint256)	11	✓
0xe610af01f92f1 ...	Collect(uint256)	1	✗
0x2ec17d1df257d ...	call()	1	✗
0x2a98d8fc14b31 ...	withdraw()	2	✓
0xa5d6accc56953 ...	CashOut(uint256)	4	✓
0x0ebe1a9cbf4e2 ...	settleEther()	2	✗
0xdd17afae8a3dd ...	Collect(uint256)	2	✗
0xb7c5c5aa4d429 ...	withdraw()	1	✓
0xf6dbe88ba55f1 ...	withdraw(uint256)	1	✓
0xaf905ab8dad7c ...	pullFunds()	1	✓

Key Takeaways

- **Novel Approach: Smart Contract Learning (Semi-supervised) (SCoolS)**
 - Tackles the lack of labeled dataset.
 - Analyzes function-level smart contract bytecode directly.
 - Speeds up analysis, enhancing detection performance.
 - Generates attack demos for developers, helping with contract exploitability testing.
- **Evaluation**
 - Measures SCoolS's performance in predicting unseen manually labeled contract functions.
 - Shows competitive performance against **three** state-of-the-art analyzers.

Thesis Summary



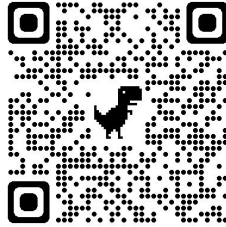
Proposal 1: DLVA
[USENIX Security 23]

Vulnerability Detection

Maintainability

Scalability

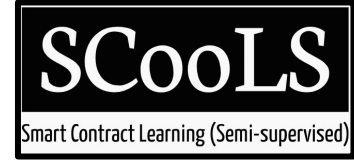
29 Vulnerabilities



Accuracy

Speed

Availability



Proposal 2: SCooLS
[IEEE BCCA 23]

Exploit Detection

Lack of labels

Localization

1 Vulnerability & Auto-exploit

Deep learning enables rapid and efficient detection of vulnerabilities in smart contracts, along with accurate exploit demonstrations. This leads to significantly improved accuracy and reduced false positives, paving the way for practical and reliable security models.

Future Work: Securing Smart Contracts

★ Ongoing work:

- **Boosting Vulnerability Detection:** Combining DLVA with LLM for deeper analysis.
- **Anomaly Tracking:** Spotting suspicious activity through transaction analysis.

★ Exploring:

- **Exploit Generation 2.0:** Leveraging transaction data for more realistic attacks.
- **Hybrid Defense:** Exploring deep learning and program analysis for enhanced security.



**Thank you for
your attention!**

Q&A